

libmr — Relazione di progetto

Laboratorio 2 A, a.a. 2025-26

Nome, cognome, matricola — da compilare

1. Architettura generale

libmr è una libreria statica (libmr.a, header pubblico include/mr.h). Il programma utente fornisce due callback (mr_mapper_t, mr_reducer_t); il framework legge l'input, esegue la pipeline di processi, raggruppa per token e scrive l'output.

Tre processi collegati da tre pipe:

- **main** (processo chiamante): legge i file, serializza le righe verso il mapper, raccoglie i record dal reducer, li ordina e li scrive sul file di output;
- **mapper**: deserializza le righe, invoca la callback mapper su thread C11, serializza le coppie <token, valore> verso il reducer;
- **reducer**: accumula tutte le coppie, raggruppa per token, invoca la callback reducer una volta per token distinto, serializza i risultati verso il main.

Input: un file regolare, oppure i file regolari contenuti **direttamente** in una directory (niente ricorsione), in **ordine lessicografico** sul nome. Ogni file è una sequenza di **righe logiche**: il '\n' non fa parte del contenuto passato al mapper; l'ultima riga può non essere terminata da '\n'. File vuoti e righe vuote sono ammessi. La serializzazione della riga sulla pipe è nella sezione 6.

I puntatori in mr_file_line_t e in mr_value_t valgono solo durante la callback: emit copia token e byte opachi prima di ritornare.

La pipeline è lineare: il main è sia sorgente delle righe sia collettore dei risultati. Sotto ogni stadio, l'ordine in cui si chiude il lato *write* (EOF sul *read* a valle). Senza (1)–(3) un lettore resta bloccato. I worker mapper **non** chiudono la pipe verso il reducer.

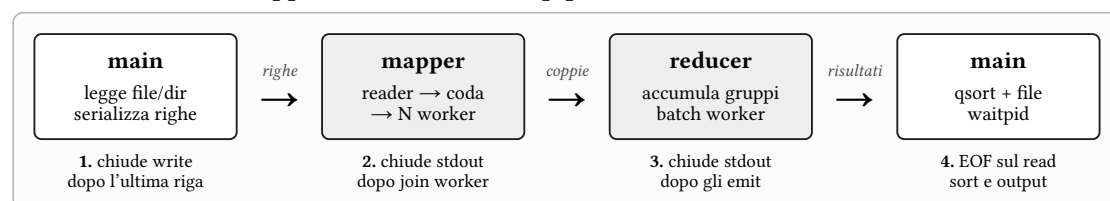


Figura 1: Pipeline di processi e chiusura delle pipe.

2. Interfaccia pubblica

Contratto minimo di mr.h (senza ricopiare l'header):

- mr_t opaco (struct mr *). mr_attr_t è un valore copiabile; dopo mr_create il chiamante può distruggerlo o modificarlo senza effetto sull'istanza.
- Ciclo: mr_attr_init → setter → mr_create → mr_start (bloccante) → mr_destroy.
- Default di mr_attr_init: mapper_threads = 1, reducer_threads = 1, queue_size = 64, log_file = NULL (file mr.log). I setter rifiutano 0 per thread e coda.
- mr_create copia configurazione, puntatori alle callback e user_arg. user_arg non è interpretato; dopo fork le modifiche nei figli non sono visibili al padre né all'altro figlio.
- Valori intermedi e risultati: byte opachi (data + size). size == 0 ammette data == NULL. Il framework non usa strlen/strcmp/printf("%s") su di essi. Il token è l'unica stringa C interpretata (alfanumerica ASCII, '\0' finale; il '\0' non sta sulla pipe).
- Ritorno: 0 successo, -1 errore, errno ove appropriato.

2.1 Errori

Errore	Quando	Effetto
EINVAL	Puntatori NULL; mapper_threads o queue_size pari a 0; secondo mr_start sullo stesso handle.	Ritorno -1, errno = EINVAL. Nessuna elaborazione avviata.
ENOMEM	Fallimento di malloc, realloc o calloc.	Ritorno -1 con errno del sistema. Cleanup di quanto già allocato.
syscall	Fallimento di pipe, fork, dup2, close, read, write, open, waitpid.	Ritorno -1 con errno del kernel. In mr_start: chiusura FD, waitpid sui figli già creati, evento sul log.
EIO	Figlio con exit diverso da 0, terminazione per segnale, protocollo corrotto o lunghezze non valide.	I figli loggano e _exit(1). Il main in waitpid imposta errno = EIO.

Scelta. Dopo un errore in mr_start, l'handle mr_t non è riutilizzabile: occorre mr_destroy e un nuovo mr_create prima di un altro mr_start.

Perché. mr_start crea processi, pipe e stato interno (record accumulati, FD). Un secondo avvio sullo stesso handle mescolerebbe descrittori e PID. Il contratto one-shot evita un reset parziale difficile da rendere corretto.

3. Processi

La sequenza di chiusura delle pipe è in figura 1. Ordine in mr_start:

1. tre pipe: main_to_mapper, mapper_to_reducer, reducer_to_main;
2. fork del mapper; nel figlio dup2 su stdin/stdout, chiusura di **tutti** i FD delle tre pipe (restano stdin/stdout già ridiretti);
3. fork del reducer; stesso schema (stdin ← read di mapper_to_reducer, stdout → write di reducer_to_main);
4. nel main restano aperti solo il write verso il mapper e il read dal reducer; gli altri sei estremi si chiudono;
5. invio righe, chiusura del write verso il mapper, raccolta risultati fino a EOF, qsort, scrittura file, waitpid di mapper e reducer.

I figli eseguono mapper_process_main / reducer_process_main e poi _exit. Nessun exec: le callback utente restano nell'immagine ereditata. I fork avvengono **prima** di qualsiasi thrd_create (nel main non ci sono thread C11 del framework).

Se il fork del reducer fallisce, il main chiude le pipe e fa waitpid del mapper già avviato. I figli in errore di dup2/close usano _exit dopo aver scritto sul log.

4. Thread C11

Header <threads.h>. Niente pthread_*. Firma dei thread: int (*)(void *).

Mapper. Un thread *reader* legge da stdin i messaggi riga, li inserisce nella coda (sezione 5) e alla EOF chiama mr_queue_close. N = mapper_threads worker fanno pop, ricostruiscono mr_file_line_t locale, invocano la callback, emettono coppie. La scrittura su stdout è sotto mtx_t: un messaggio logico non si mescola con un altro. Validazione del token (non vuoto, solo [A-Za-z0-9]) in emit. Chiusura stdout: dopo thrd_join di reader e worker (passo 2 in figura 1). Id di log: reader = 0, worker = 1...N.

Reducer. Un thread *reader* legge coppie da stdin e le inserisce nei gruppi (sezione 7). I worker **non** partono in parallelo al reader: si fa thrd_join del reader (EOF = nessuna coppia ulteriore), poi si invoca la callback sui gruppi.

Scelta. Dopo il join del reader, i gruppi si elaborano a **batch** di al più reducer_threads: per ogni batch si creano i worker (un gruppo a testa), si fa join, si passa al batch successivo.

Perché. La spec impone il reduce solo a gruppi completi: non c'è produzione concorrente di gruppi durante il reduce, quindi non serve una coda produttore-consumatore in questa fase. Il batch usa il parametro `reducer_threads` senza tenere thread idle durante tutta la lettura. Mutex su stdout anche qui: più worker dello stesso batch possono fare emit insieme.

5. Code interne

Una coda, solo nel **processo mapper**, per coordinare reader e worker. Non è una pipe e non è condivisa fra processi. Capacità = `queue_size` (elementi, non byte). Buffer circolare di puntatori a righe heap-allocate. Se piena, il reader attende `not_full`; se vuota, il worker attende `not_empty`; close sblocca i consumer e i pop successivi falliscono (fine lavoro).

```
typedef struct {
    void **items;
    size_t cap, head, tail, count;
    mtx_t mtx;
    cnd_t not_full, not_empty;
    int closed;
} mr_queue_t;
```

Scelta. Coda **bounded**: push si blocca a coda piena; `queue_size` è il numero massimo di righe in volo nel mapper.

Perché. `queue_size` è campo pubblico di `mr_attr_t` e vincolo della spec (attesa con condition variable C11). Una coda senza tetto renderebbe il parametro inerte e potrebbe far crescere la memoria se il mapper è più lento del main.

Il reducer non usa questa coda: l'accumulo è la tabella dei gruppi (sezione 7).

6. Protocollo sulle pipe

Tre tipi di messaggio, tutti length-prefixed. `readn / writen` ripetono `read/write` fino a `n` byte (gestione `EINTR` e trasferimenti parziali). Fine flusso = chiusura pipe, non un messaggio sentinella. Lunghezze in header: `int`; si rifiutano valori negativi e valori oltre i tetti sotto, **prima** della conversione a `size_t`. `token_len == 0` è invalido. Il token sulla pipe è senza `'\0'`; il ricevente alloca `token_len+1` e lo aggiunge. Payload opachi: esattamente `value_len` byte, nienti impliciti.

Scelta. Tetti: `MR_MAX_TOKEN_LEN = 1 MiB`, `MR_MAX_VALUE_LEN = 64 MiB`, `MR_MAX_LINE_LEN = 64 MiB`, `MR_MAX_NAME_LEN = 4096`.

Perché. La spec chiede limiti ragionevoli documentati. 4 KiB copre path Linux tipici; 1 MiB sul token è già oltre ogni chiave alfanumerica attesa; 64 MiB su riga e valore opaco ammette record binari grandi senza malloc da `int` non validato.

Messaggio **riga** (main → mapper):

```
typedef struct {
    int file_name_len;
    int line_len;
    unsigned long line_number;
} mr_line_header_t;
/* poi: file_name_len byte di nome, line_len byte di riga (senza '\n') */
```

Messaggio **coppia** (mapper → reducer) e **risultato** (reducer → main): stesso header; il risultato riusa il campo valore per i byte opachi emessi dal reducer.

```
typedef struct {
    int token_len;
    int value_len;
} mr_pair_header_t;
/* poi: token_len byte di token, value_len byte opachi */
```

value_len == 0: nessun payload, puntatore NULL lato ricevente.

7. Raggruppamento per token

Nel reader del reducer, ogni coppia entra in add_or_create_group. Un gruppo = un token distinto; la callback reducer parte **una volta** per gruppo, con tutti i valori, solo dopo EOF.

```
typedef struct {
    char *token;
    void **data;      /* copia heap di ogni processed_token */
    size_t *sizes;
    size_t count;
    size_t cap;
} token_group_t;
```

Lookup: scansione lineare con strcmp sul token (stringa C già terminata in ricezione). Se esiste, si accoda una **copia** dei byte; se value_size == 0, si accoda NULL. Se non esiste, realloc dell'array di gruppi e nuovo token_group_t (cap iniziale 4, raddoppio). Dopo EOF: qsort dei gruppi per token (strcmp), poi i batch della sezione 4. L'ordine sulla pipe verso il main **non** è il contratto di determinismo (i worker di uno stesso batch emettono in concorrenza): l'ordine del file è nella sezione 8.

Scelta. Tabella = array dinamico di token_group_t con lookup lineare; ordinamento dei gruppi con qsort dopo EOF.

Perché. Il reduce parte solo a input esaurito: non serve una struttura concorrente durante l'inserimento. Il token è una stringa C, quindi strcmp è lecito (non sui valori). Le copie heap rispettano il contratto di emit (il chiamante può riusare i buffer). L'array è sufficiente per il numero di chiavi distinte atteso su un corpus da laboratorio; l'ordinamento dei gruppi rende deterministica l'assegnazione ai batch.

8. Formato e ordine dell'output

Il file di output è binario, stesso layout del messaggio risultato (sezione 6): int lunghezza token, byte del token (senza '\0'), int lunghezza risultato, byte opachi del risultato. Non è testo; cat non è significativo.

Il main accumula i record in un array (record_from_reducer_t), poi scrive il file.

Scelta. I record nel file sono in ordine lessicografico sul token (strcmp). Se il reducer fa più emit per lo stesso token, l'ordine relativo nel file è l'ordine di quelle chiamate.

Perché. La spec chiede output deterministico a parità di input, callback e attributi, e di definire l'ordine se ci sono più risultati per chiave. Il sort nel main è indipendente dallo scheduling dei worker reducer: l'ordine di arrivo sulla pipe non è usato come ordine di file. Più emit sullo stesso token restano nell'ordine in cui la callback li ha prodotti (stesso worker, sequenza naturale).

9. File di log

Default: mr.log nella directory di lavoro. Apertura O_APPEND. Formato riga:

[YYYY-MM-DD HH:MM:SS] [processo] [thread_id] [evento] messaggio

- processo: main, mapper, reducer;
- thread_id: 0 per main e per i reader; 1..N per i worker;
- evento (principali): pipe, fork, thrd_start, thrd_end, close, output, stats, error;
- stats: righe inviate al mapper, coppie emesse, token distinti, record finali.

Esempio: [2026-06-21 14:30:01] [main] [0] [pipe] created 3 pipes

Eventi minimi della spec coperti: creazione pipe e processi, avvio/fine thread, apertura/chiusura file di input e output, i quattro contatori, errori.

Scelta. Mutua esclusione inter-processo con semaforo POSIX named: `sem_open("/<pid>", ...)` nel main all'apertura del log; ogni `mr_log_write` fa `sem_wait / writen / sem_post`. Il nome usa il PID del processo che crea il log, così istanze concorrenti di programmi diversi non condividono lo stesso semaforo.

Perché. Tre processi (e più thread nel mapper/reducer) scrivono lo stesso file: senza lock le righe si mescolano. Il semaforo named è ereditabile dopo `fork` (stesso mapping) e copre sia thread sia processi, come previsto dalla spec. `writen` evita righe spezzate da `write` parziali.

10. Test

`make test` compila `libmr.a` se serve, costruisce i sei eseguibili in `tests/` e li lancia in sequenza. Al primo fallimento la batteria si ferma (exit diverso da 0). I test di integrazione usano mapper e reducer propri (conteggio, byte opachi, mapper che non emette): il framework non incorpora logica di word-count.

Livello	Eseguibile	Cosa verifica
I/O e log	<code>tests/log</code>	formato riga; default <code>mr.log</code> ; scritture concorrenti da processi distinti col semaforo
	<code>tests/input</code>	file singolo; directory (ordine lessicografico); file vuoto
	<code>tests/io</code>	<code>readn/writen</code> ; messaggi riga, coppia, risultato; EOF; header/payload troncati; <code>mr_validate_len</code>
Stadi isolati	<code>tests/mapper</code>	processo mapper in un figlio: una riga in ingresso, coppia attesa su <code>stdout</code>
	<code>tests/reducer</code>	processo reducer: coppie già serializzate, raggruppamento, un <code>emit</code> per token
Integrazione	<code>tests/integration</code>	<code>mr_create/mr_start/mr_destroy</code> : word-count, directory, determinismo, valori opachi, mapper silenzioso, coda piccola con più thread, input inesistente e one-shot

Dettaglio dei comandi: README.